

Trabalho Prático 0 - Projeto e Análise de Algoritmos

Otávio Francisco Sabino Tavares

¹ Universidade Federal de Viçosa - Campus Florestal

otavio.sabino@ufv.br

1. Introdução

O presente trabalho tem como objetivo revisar a linguagem C para fazer um aquecimento e facilitar o desenvolvimento na disciplina de Projeto e Análise de Algoritmos. O tema consiste na realização de desenhos feitos com caracteres do teclado em uma moldura de 20 linhas por 80 colunas (Figura 1).

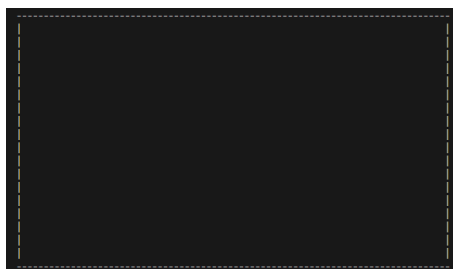


Figure 1. Moldura.

A ideia é que o usuário escolha os tipos de figura a serem gerados, considerando os 5 tipos: asteriscos simples, símbolo de '+' com asteriscos, símbolo 'x' com asteriscos, ou uma obra especial desenvolvida originalmente. Para isso, é apresentado um menu (Figura 2) para que seja escolhido o tipo, e em seguida o usuário escolhe a quantidade (de 1 a 100).

```
PROGRAMA GERADOR DE OBRA DE ARTE:
=====
Escolha o tipo de figura:
1 - Asterisco simples
2 - Símbolo de soma com asteriscos
3 - Letra x com asteriscos
4 - Figuras misturadas
Outro numero - Obra de arte especial
2
escolha o numero (de 1 a 100): █
```

Figure 2. Menu.

Em seguida, com base nas entradas do usuário, o sistema gera posições aleatórias dentro da moldura para as figuras (nas opções de 1 a 4) ou gera aleatoriamente um número referente a alguma das formações (desenvolvidas na obra especial a ser especificada futuramente).

2. Organização do projeto

No projeto, (Figura 3), temos a pasta "include" com os arquivos de cabeçalho (.h) referentes a códigos auxiliares; na pasta "src" temos os arquivos principais (.c) com as principais funções, sendo "main.c" o código que executa as declarações dos arquivos auxiliares ("uteis.c", com algumas funções importantes para o funcionamento geral, e "bancoFormacoes.c" com o desenvolvimento da obra especial). Dentro da pasta do projeto, temos também a presente documentação, o executável e o Makefile para facilitar a compilação.

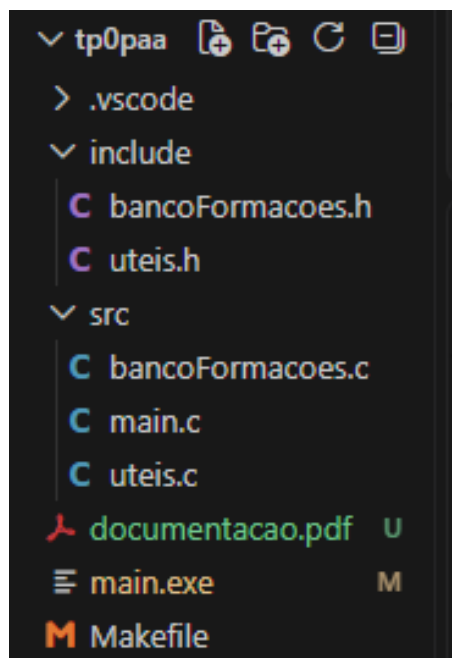


Figure 3. Disposição dos arquivos.

3. Desenvolvimento

3.1. Uso da matriz

A estratégia escolhida para a execução do trabalho foi o uso de uma matriz, inicializada a partir de uma função (Figura 4) no arquivo "uteis.c", com as bordas com caracteres '-' e '—', e os demais sendo preenchidos com espaços em branco (' ').

```

void criaMatrizVazia(char caracteres[20][80]){
    for(int i = 0; i < 80; i++){
        caracteres[0][i] = '-';
        caracteres[19][i] = '-';

        if(i < 19 && i > 0){
            caracteres[i][0] = '|';
            caracteres[i][79] = '|';
        }
    }

    for(int i = 1; i < 19; i++){
        for(int j = 1; j < 79; j++){
            caracteres[i][j] = ' ';
        }
    }
}

```

Figure 4. Criação de matriz.

3.2. Leitura da entrada

No momento em que o usuário digita o número referente à quantidade de figuras, são feitas algumas verificações para garantir que foi digitado um valor inteiro e garantir ainda que trabalhem com um valor de 1 a 100.

```

int leituraNumero(){
    printf("escolha o numero (de 1 a 100): ");
    int n;

    if (scanf("%d", &n) != 1) {
        printf("Entrada invalida");
        exit(1);
    }

    if(n > 100) n = 100;

    if(n < 1) n = 1;

    return n;
}

```

Figure 5. Função de leitura.

3.3. Asteriscos simples

Nesta etapa, foi desenvolvida uma função que gera uma linha e uma coluna aleatórias até que a posição gerada seja inédita, ou seja, ainda não haja asterisco na exata posição. O asterisco é então posicionado na matriz.

```

void asteriscoSimples(char caracteres[20][80]){
    int linha;
    int coluna;

    do{
        linha = (rand() % 18) + 1;
        coluna = (rand() % 78) + 1;
    }while(caracteres[linha][coluna] == '*');

    caracteres[linha][coluna] = '*';
}

```

Figure 6. Função que gera os asteriscos simples.

3.4. Símbolo de soma

Aqui, é usada uma função que gera uma linha e uma coluna aleatórias e verifica-se as posições adjacentes, para que nenhum asterisco deste símbolo (Figura 7) coincida com asteriscos de outra figura (Figura 8).

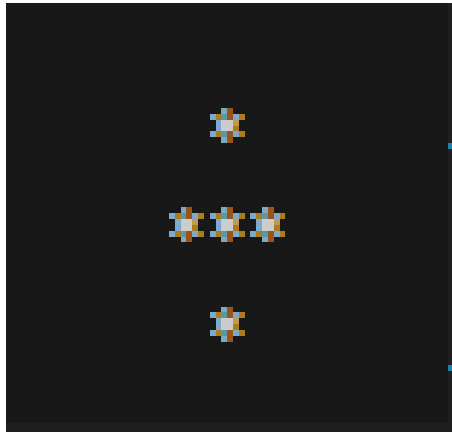


Figure 7. Criação do '+’.

```
void soma(char caracteres[20][80]){
    int linha;
    int coluna;

    do{
        linha = (rand() % 16) + 2;
        coluna = (rand() % 76) + 2;

        //condicoes para que as posicoes nao se repitam
        if(caracteres[linha][coluna] == '*'){
            continue;
        }

        if(caracteres[linha + 1][coluna] == '*' || caracteres[linha - 1][coluna] == '*'){
            continue;
        }

        if(caracteres[linha][coluna + 1] == '*' || caracteres[linha][coluna - 1] == '*'){
            continue;
        }
    }
}
```

Figure 8. Geração da posição e primeiras verificações.

Observação: é possível que no Linux a esse símbolo seja impresso de forma um pouco diferente (Figura 9), mas é apenas uma variação visual.

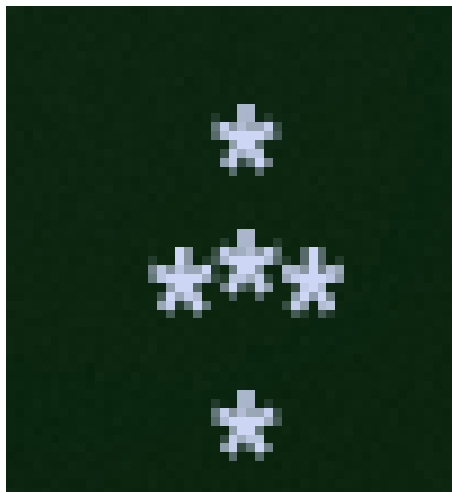


Figure 9. Símbolo '+’ no Linux.

Além disso, também são feitas verificações (Figura 10) para que haja ao menos um caractere de distância entre outras figuras e a próxima a ser gerada, ou seja, para que os símbolos de soma não se "encaixem", dificultando a visualização.

```
//condições para que as figuras não se misturem
if(caracteres[linha + 1][coluna + 1] == '*' || caracteres[linha + 1][coluna - 1] == '*'){
    continue;
}

if(caracteres[linha - 1][coluna - 1] == '*' || caracteres[linha - 1][coluna + 1] == '*'){
    continue;
}
```

Figure 10. Verificações finais.

Note que é usado o comando "continue". Isso se dá porque foi feito um "while infinito", de forma que, se uma dessas verificações for verdadeira, o laço faz mais uma interação. Ele só para com o "break" (Figura 11) caso nenhuma das condições aconteça, ou seja, podemos criar a figura com base na linha e coluna sorteadas dessa vez.

```
break;
}while(1);

caracteres[linha][coluna] = '*';
caracteres[linha+1][coluna] = '*';
caracteres[linha-1][coluna] = '*';
caracteres[linha][coluna + 1] = '*';
caracteres[linha][coluna - 1] = '*';
```

Figure 11. Criação do '+'.

3.5. Símbolo de 'x'

Esta operação é semelhante à anterior. O símbolo (Figura 12) é gerado com verificações semelhantes ao '+', para não coincidir nem se misturar com outras figuras.

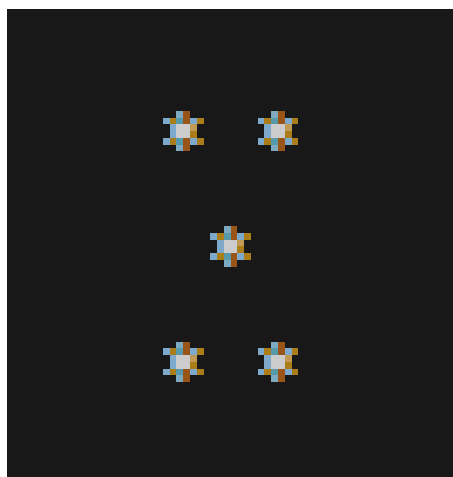


Figure 12. Símbolo x.

3.6. Execução na main

Para executar os itens anteriores na main, inicialmente é usada a função de criar a matriz, e é feita a leitura da entrada (Figura 13).

```

int main(){
    char caracteres[20][80];
    criaMatrizVazia(caracteres);
    printf("PROGRAMA GERADOR DE OBRA DE ARTE:\n");
    printf("-----\n");
    printf("Escolha o tipo de figura:\n1 - Asterisco simples\n2 - Símbolo de soma com asteriscos\n3 - Letra x com asteriscos\n4 - Figuras misturadas\nOutro numero - Obra de arte especial\n");
    printf("\n");
    int op;
    if (scanf("%d", &op) != 1) {
        printf("Entrada inválida");
        exit(1);
    }
    int n = leituraNumero();
}

```

Figure 13. Criação da matriz e entrada do usuário.

Após isso, é preciso usar a biblioteca "time", para que sejam gerados números aleatórios diferentes a cada execução (Figura 14).

```

//gerar numeros diferentes a cada execução
srand(time(NULL));

```

Figure 14. Uso da biblioteca.

Assim, podemos, com base nas entradas, gerar as figuras, como no exemplo (Figura 15) para asteriscos simples, com base na quantidade digitada pelo terminal.

```

switch(op){
    case 1:
        for(int i = 0; i < n; i++){
            asteriscoSimples(caracteres);
        }
        imprimeMatriz(caracteres);
        break;
}

```

Figure 15. Geração dos símbolos.

Neste momento, também é usada a função de impressão da matriz (Figura 16), no arquivo "uteis.c", que é apenas uma função para percorrer a matriz exibindo no terminal caractere por caractere.

```

void imprimeMatriz(char caracteres[20][80]){
    for(int i = 0; i < 20; i++){
        for(int j = 0; j < 80; j++){
            printf("%c", caracteres[i][j]);
        }
        printf("\n");
    }
}

```

Figure 16. Função de impressão.

3.7. Figuras aleatórias

Na opção 4, devemos então misturar as figuras anteriores. Para isso, foi criada outra função (Figura 17), que gera um número aleatório (0, 1 ou 2): para cada iteração referente à quantidade digitada pelo usuário, se for sorteado 0, é gerado um asterisco simples, se for 1 é gerado um '+', e se for 2 é gerado o 'x'. São usadas as mesmas funções já mostradas.

```

void aleatorio(char caracteres[20][80], int n){
    for(int i = 0; i < n; i++){
        int nFigura = rand() % 3;
        switch(nFigura){
            case 0:
                asteriscoSimples(caracteres);
                break;

            case 1:
                soma(caracteres);
                break;

            case 2:
                x(caracteres);
                break;

            default:
                break;
        }
    }
}

```

Figure 17. Geração aleatória.

3.8. Obra especial

Nesta etapa, a ideia foi fazer inicialmente um campo de futebol fixo (como na Figura 18, com o exemplo da área), e fazer funções referentes a 7 formações comuns no esporte (4-3-3, 4-2-3-1, 4-5-1, 3-4-3, 3-5-2 e uma variação do 4-5-1), com os jogadores representados na escalação.

```

void campoFutebol(char caracteres[20][80], int n){
    //área
    for(int i = 1; i < 12; i++){
        caracteres[6][i] = '-';
        caracteres[14][i] = '-';
    }
    for(int i = 1; i < 22; i++){
        caracteres[3][i] = '-';
        caracteres[16][i] = '-';
    }
    for(int i = 7; i < 14; i++){
        caracteres[i][11] = '|';
    }
    for(int i = 4; i < 17; i++){
        caracteres[i][22] = '|';
    }

    //penalti
    caracteres[10][17] = '*';
}

```

Figure 18. Exemplo da área.

A geração aleatória do número consiste em fazer um cálculo com dois números aleatórios e o número digitado pelo usuário e a partir disso obter um número de 0 a 6, correspondente a uma das formações (como as primeiras formações na Figura 19).

```

int a = rand() % 6 + 1;
int b = rand() % 7;

int formacao = (a * n + b) % 7;

switch(formacao){
    case 0:
        f442(caracteres);
        break;

    case 1:
        f433(caracteres);
        break;

    case 2:
        f4231(caracteres);
        break;
}

```

Figure 19. Sorteio das formações.

Assim, é inicialmente projetado o campo, primeiro planejado em um bloco de notas, e com base nisso, desenhado na função "campoFutebol" no arquivo "uteis.c". Dentro dessa função, porém, são usadas funções referentes a cada uma das formações; elas se encontram no arquivo "bancoFormacoes.c". As formações são construídas com base em suas características. Por exemplo, na formação 4-4-2, temos como características: uma linha de 4 jogadores na defesa, dois meio-campistas, dois alas, e dois atacantes, e é com base nisso que ela é construída (Figura 20).

```

void f442(char caracteres[20][80]){
    printf("==Formação 4-4-2==\n");
    linhaDe4(caracteres);

    doisVolantes(caracteres);

    alas(caracteres);

    doisAtacantes(caracteres);
}

```

Figure 20. Construção da formação 4-4-2.

Observe que são usadas ainda algumas funções como "linhaDe4" (Figura 21), que corresponde à construção da linha de 4 defensores; esse tipo de função é feita para características comuns a diferentes formações.


```

void linhaDe4(char caracteres[20][80]){
    caracteres[2][20] = '(';
    caracteres[2][21] = '6';
    caracteres[2][22] = ')';

    caracteres[7][14] = '(';
    caracteres[7][15] = '4';
    caracteres[7][16] = ')';

    caracteres[13][14] = '(';
    caracteres[13][15] = '3';
    caracteres[13][16] = ')';

    caracteres[17][20] = '(';
    caracteres[17][21] = '2';
    caracteres[17][22] = ')';
}

```

Figure 21. Linha de 4 defensores.

Assim, a construção de cada formação (salvo raras exceções) consiste em combinar as funções com base na disposição de jogadores.

Além disso, como curiosidade, é feito um vetor para cada formação com alguns times marcantes que a usaram, e a cada execução é sorteada uma dessas equipes para ser lembrada junto com o desenho (Figura 22).

```

char times[4][30] = {"Atletico de Madrid de Simeone", "Manchester United de Ferguson", "Brasil de 1994",
                    "Milan de Ancelotti"};

imprimeMatriz(caracteres);
printf("Exemplo de time que usou: ");
int ex = rand() % 4;
printf("%s\n", times[ex]);

```

Figure 22. Vetor de times.

Ou seja, o fluxo da obra especial consiste em: ler o número do usuário, desenhar o campo base, sortear um número com base nos aleatórios gerados e no digitado pelo terminal, obter um número correspondente a uma das formações, desenhar a formação e sortear um time correspondente para ser mostrado no terminal como curiosidade.

4. Compilação e execução

Para compilar o código no Windows ou Linux, é possível digitar o comando "make" para usar o Makefile, ou o comando completo "gcc src/uteis.c src/bancoFormacoes.c src/main.c -o main". Para executar, basta digitar "./main".

5. Resultados

É possível observar alguns testes para cada funcionalidade.

5.1. Asterisco simples

Exemplo para 20 asteriscos:

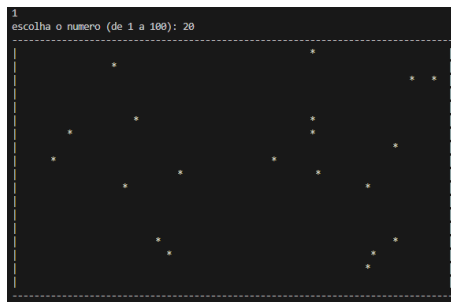


Figure 23. 20 asteriscos.

Exemplo para 97 asteriscos:

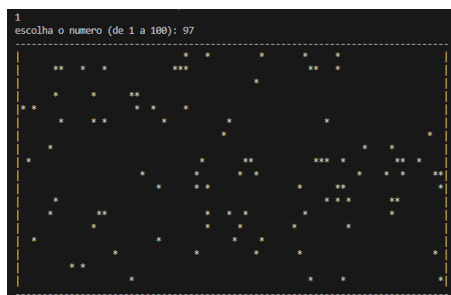


Figure 24. 97 asteriscos.

5.2. Símbolo '+'

Exemplo para 23 símbolos:



Figure 25. 23 '+'.

Exemplo para 90 símbolos:

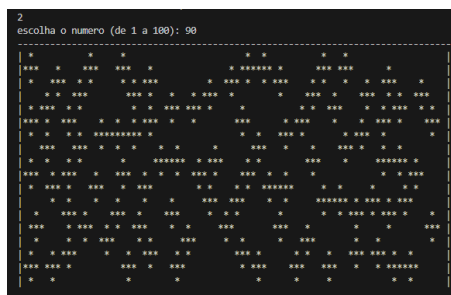


Figure 26. 90 '+'.

5.3. Símbolo 'x'

Exemplo para 25 símbolos:

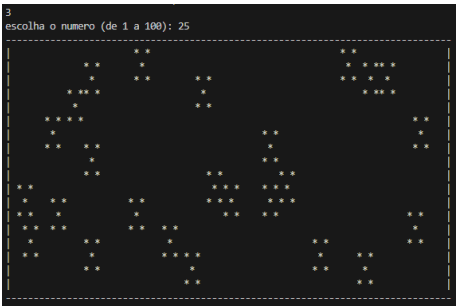


Figure 27. 25 'x'.

Exemplo para 93 símbolos:

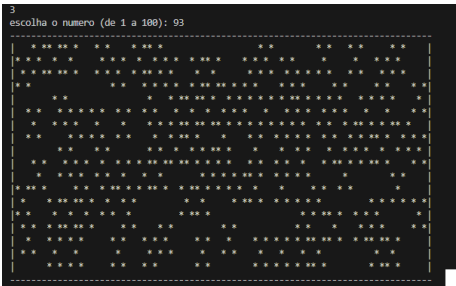


Figure 28. 97 'x'.

5.4. Figuras aleatórias

Exemplo para 19 símbolos:

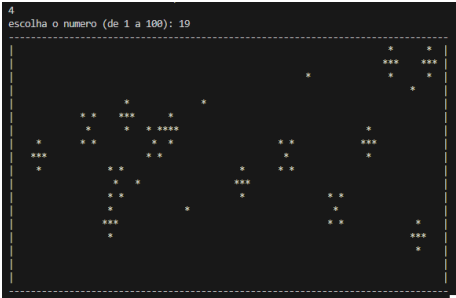


Figure 29. 19 figuras.

Exemplo para 87 símbolos:

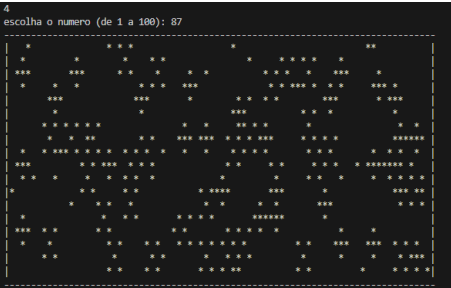


Figure 30. 87 figuras.

5.5. Obra especial

Exemplo 1:

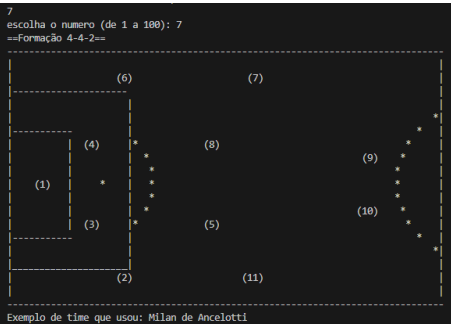


Figure 31. Exemplo 1.

Exemplo 2:

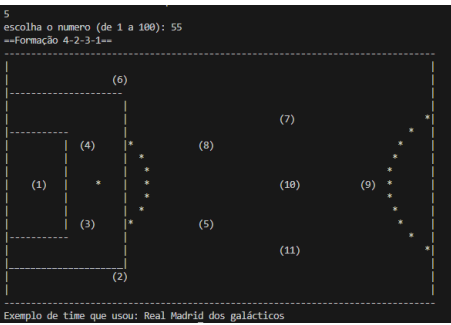


Figure 32. Exemplo 2.

Exemplo 3:

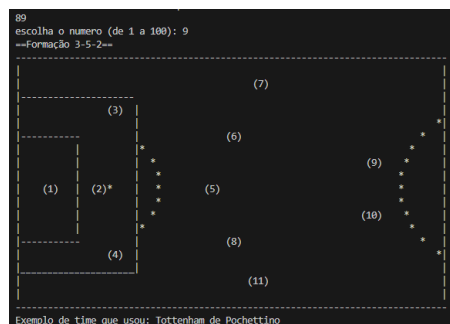


Figure 33. Exemplo 3.

6. Conclusão

Este trabalho foi útil para relembrar a linguagem C e suas características, especialmente após bastante tempo sem programar nela, sendo possível rever coisas importantes como matrizes e vetores, uso de números aleatórios, laços de repetição, entre outros, o que é importante visto o objetivo do trabalho.

7. Referências

W3Schools. *C Random Numbers*. Disponível em: https://www.w3schools.com/c/c_random_numbers.php.

Coaches' voice. *Sistemas de jogo: 4-3-3*. Disponível em: <https://es.coachesvoice.com/cv/analise-tatica-sistemas-de-jogo-4-3-3/>.

Coaches' voice. *Sistemas táticos: 4-4-2*. Disponível em: <https://es.coachesvoice.com/cv/4-4-2-formacao-tatica/>.

Lance. *4-2-3-1: as vantagens e desvantagens do esquema tático do futebol*. Disponível em: <https://www.lance.com.br/lancepedia/4-2-3-1-esquema-futebol-vantagens-desvantagens.html>.

Lance *3-5-2: as vantagens e desvantagens do esquema tático do futebol*. Disponível em: <https://www.lance.com.br/lancepedia/3-5-2-esquema-futebol-vantagens-desvantagens.html>.